



La forêt interdite

Mini RPG Python - cours, exemples et entraînements

Mode d'emploi

Lis chaque notion, prédis le résultat, puis vérifie-le sur le site ou dans ton éditeur Python. Les résultats attendus concernent les exemples de cours. Les exercices restent à compléter par l'élève.

Sommaire

- Partie 1 - Créer son personnage
- Partie 2 - Hasard, bibliothèques et premières fonctions
- Partie 3 - Explorer avec une boucle for
- Partie 4 - Faire durer le combat avec while
- Partie 5 - Gérer les monstres avec une liste
- Partie 6 - Construire le bestiaire avec des dictionnaires

PARTIE 1 - 5 POINTS

Créer son personnage

Saisir des informations, calculer et prendre une décision.

1. Une variable garde une valeur

Une variable est un nom associé à une valeur.

Pour créer ou modifier cette association, on utilise `=`.

Python exécute les lignes dans l'ordre : la dernière affectation remplace la précédente.

```
pv = 50

pv = pv - 10

print(pv)
```

Résultat attendu

40

À surveiller

`=` signifie « affecter », pas « est égal à » au sens d'un test. On calcule le côté droit avant de modifier la variable.

2. Texte et nombres ne se manipulent pas pareil

Une chaîne de caractères (str) se met entre guillemets.

Un entier (int) s'écrit sans guillemets.

`+` additionne des entiers ; entre deux chaînes, il colle les textes.

```
print(5 + 5)

print("5" + "5")
```

Résultat attendu

10
55

À surveiller

La valeur 5 et le texte « 5 » se ressemblent à l'écran, mais n'ont pas le même type.

3. Lire avec input, afficher avec print

input affiche une question, puis renvoie le texte tapé.

print affiche une ou plusieurs valeurs.

Les virgules dans print séparent les éléments à afficher.

```
nom = input("Ton nom ? ")  
  
print("Bienvenue", nom)
```

Résultat attendu

Si tu saisis Lina : Bienvenue Lina

À surveiller

input renvoie toujours du texte. Écrire input sans les parenthèses n'appelle pas la fonction.

4. Convertir une saisie numérique

Pour calculer avec un nombre saisi, int transforme le texte en entier.

Lis de l'intérieur vers l'extérieur : input récupère le texte, int le convertit, puis = affecte le résultat.

```
force = int(input("Force ? "))  
  
print(force + 2)
```

Résultat attendu

Si tu saisis 5 : 7

À surveiller

Pour cette séance, on suppose que l'élève tape un entier. Si on tape cinq en lettres, int provoque une erreur ; ce cas n'est pas demandé.

5. Choisir un chemin avec if, elif et else

`==` compare deux valeurs.

if exécute un bloc si son test est vrai ; elif teste un autre cas ; else traite les autres possibilités.

Les quatre espaces au début d'une ligne montrent le bloc auquel elle appartient.

```
action = "dormir"

if action == "attaquer":
    print("À l'attaque !")
elif action == "soigner":
    print("Une potion !")
else:
    print("Action inconnue")
```

Résultat attendu

Action inconnue

À surveiller

Ne remplace pas `elif action == "soigner"` par `else` : sinon dormir déclencherait aussi un soin.

6. Vérifier plusieurs contraintes

`and` signifie que toutes les conditions reliées doivent être vraies.

Pour valider le personnage, la somme doit être 15 ET chaque compétence doit être positive ou nulle.

```
points = 7

valide = points >= 0 and points <= 15

print(valide)
```

Résultat attendu

True

À surveiller

Une somme correcte ne suffit pas : $-1 + 8 + 8$ vaut 15, mais une compétence négative doit être refusée.

Exercices intermédiaires

1. Échauffement A · la potion timide

Le héros a 30 PV. Ajoute 12 PV puis affiche sa nouvelle vie. Écris les deux lignes manquantes.

```
pv = 30

# TODO : augmenter les PV de 12

# TODO : afficher les PV
```

Objectif de vérification

42

Indice : Réutilise `pv = pv + ...` puis `print(...)`.

2. Échauffement B · une force bien réelle

Demande la force sous forme d'un entier, puis affiche le double. Teste avec 4 puis 7.

```
# TODO : demander et convertir la force

# TODO : afficher son double
```

Objectif de vérification

4 → 8 ; 7 → 14

Indice : Utilise `int(input(...))` ; le double se calcule avec `* 2`.

Mission à réaliser

Ce que tu dois obtenir

Une création de personnage en console. On ne combat pas encore dans le programme à rendre de cette partie.

1. Dans Mon travail, écris `nom = input(...)` et `classe = input(...)`. Les trois classes reconnues sont exactement guerrier, mage et archer, en minuscules.
2. Avec `if / elif / else`, affiche un accueil pour une classe reconnue et « Classe inconnue » sinon. Tu n'as pas à redemander la classe dans cette partie.
3. Demande force, intelligence et agilité avec `int(input(...))`. Les saisies sont supposées être des entiers.
4. Calcule `total = force + intelligence + agilité`. Affiche le nom, la classe, les trois valeurs et le total.
5. Affiche « Répartition valide » seulement si total vaut 15 ET si aucune des trois valeurs n'est négative. Sinon, affiche « Répartition invalide ». Le contrôle de la répartition est indépendant du contrôle de classe.
6. Lance les tests proposés et ajoute deux commentaires : entrées, résultat attendu, résultat observé. Télécharge ensuite ton fichier .py.

Tests à effectuer

- Lina, mage, 5, 4, 6 → total 15 et répartition valide.
- 5, 5, 6 → total 16, invalide ; -1, 8, 8 → total 15 mais invalide.
- Classe dragon → classe inconnue ; dormir dans l'exemple → PV inchangés.

Hasard, bibliothèques et premières fonctions

Créer puis réutiliser plusieurs fonctions dans une rencontre aléatoire.

1. Utiliser un module

Un module regroupe des fonctions.

import random permet d'utiliser celles du hasard.

On écrit le nom du module, un point, puis le nom de la fonction.

```
import random

de = random.randint(1, 20)

print(de)
```

Résultat attendu

Un entier entre 1 et 20, bornes incluses.

À surveiller

Ne nomme pas ton fichier random.py : il risquerait de masquer le module fourni avec Python.

2. Comprendre les bornes du hasard

random.random() donne un nombre supérieur ou égal à 0 et strictement inférieur à 1.

random.randint(a, b) donne un entier entre a et b inclus.

Le hasard est produit par un algorithme : il est pseudo-aléatoire.

```
import random

print(random.random())

print(random.randint(0, 10))
```

Résultat attendu

Par exemple 0.37 puis 8. Le résultat peut changer.

À surveiller

int(random.random()*10) donne 0 à 9, pas 0 à 10. Une petite série de tirages n'a pas forcément les proportions prévues.

3. Une fonction sans paramètre

Une fonction peut ne recevoir aucune information.

Les parenthèses restent obligatoires dans la définition et dans l'appel.

Ici chaque appel effectue un nouveau tirage.

```
import random

def lancer_piece():

    return random.choice(["pile", "face"])

print(lancer_piece())
```

Résultat attendu

pile ou face

À surveiller

Écrire `print(lancer_piece)` affiche une représentation de la fonction. Il faut les parenthèses pour l'appeler.

4. Paramètres et arguments

Un paramètre est un nom utilisé dans la définition.

Un argument est une valeur fournie pendant l'appel.

Une même fonction devient réutilisable avec plusieurs arguments.

```
def calculer_degats(force, bonus):

    return force + bonus

print(calculer_degats(8, 3))

print(calculer_degats(12, 0))
```

Résultat attendu

11
12

À surveiller

L'ordre des arguments suit celui des paramètres. `calculer_degats(8)` provoque une erreur car il manque le bonus.

5. Fabriquer un dé réutilisable

Une fonction peut utiliser un paramètre dans l'appel d'une fonction de bibliothèque.

`lancer_de(6)` et `lancer_de(20)` exécutent le même algorithme avec des bornes différentes.

```
import random

def lancer_de(nb_faces):

    return random.randint(1, nb_faces)

print(lancer_de(6))

print(lancer_de(20))
```

Résultat attendu

Un entier de 1 à 6, puis un entier de 1 à 20.

À surveiller

La fonction renvoie le tirage. Elle ne doit pas toujours utiliser 20 à la place du paramètre `nb_faces`.

6. Renvoyer avec return

return transmet un résultat au code qui a appelé la fonction et termine cet appel.

print montre du texte à l'écran.

Si tu veux réutiliser une valeur dans un calcul ou un test, il faut la renvoyer.

```
def bonus_affiche(force):  
    print(force + 2)  
  
def bonus_renvoie(force):  
    return force + 2  
  
bonus_affiche(5)  
  
attaque = bonus_renvoie(5)  
  
print(attaque + 1)
```

Résultat attendu

7
8

À surveiller

Le premier 7 est seulement affiché. Le second 7 est d'abord renvoyé, stocké, puis réutilisé pour calculer 8.

7. Une fonction peut renvoyer un booléen

Une comparaison produit True ou False.

La fonction peut renvoyer directement cette comparaison, puis son résultat peut contrôler un if.

```
def attaque_reussie(de, seuil):  
    return de >= seuil  
  
if attaque_reussie(11, 11):  
    print("Touché !")
```

Résultat attendu

Touché !

À surveiller

Choisis >= ou > selon la règle. Teste toujours juste avant, exactement sur, puis juste après le seuil.

8. Tester avant de remettre le hasard

Pour vérifier une règle, commence avec des valeurs choisies.

Vérifie les frontières, puis remets le tirage aléatoire.

Un petit nombre de tirages n'a pas forcément les proportions théoriques.

```
def attaque_reussie(de, seuil):  
    return de >= seuil  
  
print(attaque_reussie(10, 11))  
print(attaque_reussie(11, 11))  
print(attaque_reussie(12, 11))
```

Résultat attendu

False
True
True

À surveiller

Un test aléatoire peut réussir par hasard malgré une erreur. Les valeurs fixes rendent le test reproductible.

9. Faire entrer une valeur dans une fonction

input lit toujours du texte.

Convertis la saisie, puis donne la valeur obtenue comme argument de la fonction.

Tu peux changer la réponse ci-dessous avant chaque exécution.

```
import random  
  
def lancer_de(nb_faces):  
    return random.randint(1, nb_faces)  
  
faces = int(input("Nombre de faces ? "))  
resultat = lancer_de(faces)  
print("Résultat :", resultat)
```

Résultat attendu

Avec 6 : un entier entre 1 et 6.

À surveiller

La valeur saisie doit être un entier positif. Dans cet exercice, on ne traite pas encore une saisie non numérique.

Exercices intermédiaires

1. Échauffement A · le dé miniature

Importe random et lance un dé à 6 faces. Affiche son résultat.

```
# TODO : import

# TODO : tirer un entier entre 1 et 6

# TODO : afficher
```

Objectif de vérification

Toujours un entier de 1 à 6.

Indice : La borne supérieure de randint est incluse.

2. Échauffement B · trois points de courage

Écris une fonction ajouter_bonus(points) qui renvoie points + 3. Appelle-la avec 4 et affiche le résultat.

```
def ajouter_bonus(points):

    # TODO : renvoyer le résultat

    pass

print(ajouter_bonus(4))
```

Objectif de vérification

7

Indice : Remplace pass par return suivi du calcul.

3. Entraînement C · un dé à plusieurs faces

Complète lancer_de(nb_faces), puis appelle-la avec 6, 10 et 20. Chaque résultat doit respecter sa borne.

```
import random

def lancer_de(nb_faces):

    # TODO : utiliser nb_faces

    pass

print(lancer_de(6))

print(lancer_de(10))

print(lancer_de(20))
```

Objectif de vérification

Trois entiers : 1–6, puis 1–10, puis 1–20.

Indice : Utilise random.randint(1, nb_faces), puis return.

4. Entraînement D · print ou return ?

Corrige bonus : le programme doit pouvoir calculer et afficher 10. Ne modifie pas la dernière ligne.

```
def bonus(force):

    print(force + 2)

resultat = bonus(7)

print(resultat + 1)
```

Objectif de vérification

10

Indice : La fonction doit renvoyer 9 au lieu de seulement l'afficher.

Mission à réaliser

Ce que tu dois obtenir

Une surprise aléatoire, trois fonctions réutilisables et une petite rencontre qui assemble leurs résultats. Tkinter reste facultatif.

1. Importe random au début de Mon travail.
2. Tire chance avec random.random(). Affiche « Attaque surprise » si chance < 0.5, sinon « Aucun ennemi ».

3. Écris `lancer_de(nb_faces)`, qui renvoie un entier entre 1 et `nb_faces`. Appelle-la avec 6 puis 20.
4. Écris `attaque_reussie(de, seuil)`, qui renvoie le booléen de `>= seuil`. Affiche ses résultats pour (10, 11), (11, 11) et (12, 11).
5. Écris `calculer_degats(force, bonus)`, qui renvoie la somme de ses deux paramètres. Vérifie `calculer_degats(8, 3)`.
6. Assemble les fonctions : lance un dé à 20 faces, teste une réussite au seuil 11 et affiche les dégâts calculés seulement si l'attaque réussit.
7. Ajoute trois commentaires expliquant définition/appel, paramètre/argument et print/return. Exporte le `.py`.

Tests à effectuer

- `chance = 0.49` → surprise ; `chance = 0.5` → aucun ennemi.
- `lancer_de(6)` reste entre 1 et 6 ; `lancer_de(20)` reste entre 1 et 20.
- `attaque_reussie(10, 11)` → False ; `attaque_reussie(11, 11)` → True.
- `calculer_degats(8, 3)` → 11 ; le résultat est réutilisable.

Explorer avec une boucle for

Répéter un nombre connu de fois et compter des rencontres.

1. L'indentation change le sens

Les lignes indentées appartiennent au bloc situé au-dessus.

Une ligne revenue à gauche est exécutée après ce bloc, quel que soit le chemin pris dans le if.

```
potions = 0

if potions == 0:
    print("Repos")
else:
    print("Boire")

print("Départ")
```

Résultat attendu

Repos
Départ

À surveiller

Si Départ est indenté dans else, il ne s'affiche plus lorsque potions vaut 0.

2. Répéter avec for et range

for fait prendre successivement plusieurs valeurs à une variable.

Avec range(début, fin), le début est inclus et la fin exclue.

Chaque valeur déclenche une exécution du bloc indenté.

```
for numero in range(1, 4):
    print("Pas", numero)

print("Arrivée")
```

Résultat attendu

Pas 1
Pas 2
Pas 3
Arrivée

À surveiller

Pour afficher les nombres 1 à 10, il faut range(1, 11). range(10) donne 0 à 9.

3. Compter sans tout remettre à zéro

Initialise un compteur avant la boucle.

Augmente-le dans la boucle lorsque l'événement se produit.

Affiche le bilan après la boucle pour l'obtenir une seule fois.

```
compteur = 0

for i in range(3):

    compteur = compteur + 1

print(compteur)
```

Résultat attendu

3

À surveiller

Placer compteur = 0 dans la boucle efface le total précédent à chaque tour.

4. Découper un tirage en intervalles

if / elif / else sélectionne un seul cas.

Le elif n'est testé que si le if était faux : il n'est donc pas nécessaire de répéter la borne basse.

```
x = 0.6

if x < 0.5:

    print("A")

elif x < 0.8:

    print("B")

else:

    print("C")
```

Résultat attendu

B

À surveiller

Les intervalles sont [0 ; 0.5[, [0.5 ; 0.8[et [0.8 ; 1[. Ils représentent 50 %, 30 % et 20 % des tirages, en théorie.

5. Appeler une fonction dans une boucle

La fonction écrite une fois peut être appelée à chaque tour.

La boucle organise la répétition ; la fonction garde un rôle précis.

Cette séparation évite de recopier le même if plusieurs fois.

```
def doubler(n):  
    return 2 * n  
  
for valeur in range(1, 4):  
    print(doubler(valeur))
```

Résultat attendu

```
2  
4  
6
```

À surveiller

Défins la fonction avant la boucle. Dans le RPG, choisir_ennemi reçoit un nouveau tirage à chaque tour.

6. Choisir le nombre de répétitions avec input

Une saisie convertie en int peut fixer le nombre de tours.

range(1, nombre + 1) permet alors d'afficher exactement les numéros de 1 à nombre.

```
nombre = int(input("Combien de rencontres ? "))  
  
for numero in range(1, nombre + 1):  
    print("Rencontre", numero)
```

Résultat attendu

```
Avec 3 :  
Rencontre 1  
Rencontre 2  
Rencontre 3
```

À surveiller

Teste 1 puis 0. Avec 0, le parcours est vide et aucun numéro ne s'affiche.

Exercices intermédiaires

1. Échauffement A · trois portes

Affiche Porte 1, Porte 2, Porte 3 avec une boucle. Ajoute une ligne Fin affichée une seule fois.

```
# TODO : une boucle et son affichage

# TODO : Fin après la boucle
```

Objectif de vérification

Porte 1
Porte 2
Porte 3
Fin

Indice : range(1, 4) ; Fin reste sans indentation.

2. Échauffement B · une bourse qui se remplit

Commence avec 0 pièce. Gagne 2 pièces à chacun des 4 tours. Affiche seulement le total final.

```
pieces = 0

for tour in range(4):

    # TODO : augmenter la bourse

    pass

print(pieces)
```

Objectif de vérification

8

Indice : Augmente pieces dans le bloc ; ne remets pas pieces à 0 à chaque tour.

Mission à réaliser

Ce que tu dois obtenir

L'exercice d'indentation suivi d'une exploration de dix rencontres, avec un bilan final.

1. Recopie l'exemple sur les potions dans Mon travail. Déplace seulement l'indentation de la ligne « Entrer dans la grotte » pour qu'elle soit exécutée uniquement lorsqu'on boit une potion.
2. Définis choisir_ennemi(x). Elle renvoie gobelin si $x < 0.5$, araignee si $x < 0.8$ après le premier test, troll sinon. Utilise if / elif / else.
3. Initialise nombre_gobelins à 0 avant la boucle des rencontres.
4. Écris une boucle de 1 à 10 inclus. À chaque tour, tire x avec random.random(), appelle choisir_ennemi(x), puis affiche le numéro et le nom de l'ennemi.
5. Si l'ennemi vaut gobelin, augmente le compteur. Après la boucle, affiche le nombre de gobelins une seule fois.
6. Teste les frontières de la fonction, puis remplace temporairement le tirage par 0.2 : tu dois compter 10 gobelins. Remets le hasard et exporte.

Tests à effectuer

- 0 et 0.49 → gobelin ; 0.5 et 0.79 → araignee ; 0.8 et 0.99 → troll.
- Exactement dix lignes de rencontre, puis un bilan ; le compteur vaut entre 0 et 10.
- En remplaçant tous les tirages par 0.2, le bilan doit compter 10 gobelins.

PARTIE 4 - 5 POINTS

Faire durer le combat avec while

Répéter jusqu'à une condition de fin et gérer un tour complet.

1. Répéter tant qu'une condition est vraie

while vérifie une condition avant chaque tour.

Si elle est vraie, le bloc s'exécute puis le test recommence.

Si elle est fausse dès le départ, le bloc ne s'exécute jamais.

```
vie = 30

while vie > 0:

    vie = vie - 10

    print(vie)
```

Résultat attendu

```
20
10
0
```

À surveiller

Sans mise à jour de vie, cette boucle pourrait ne jamais se terminer. Le bouton Arrêter permet de reprendre la main.

2. Deux survivants sont nécessaires au combat

and exige deux conditions vraies à la fois.

Le combat doit continuer si le héros ET le monstre sont vivants.

or suffirait si un seul était vivant : ce n'est pas la bonne règle.

```
pv_hero = 0

pv_monstre = 40

print(pv_hero > 0 and pv_monstre > 0)
```

Résultat attendu

```
False
```

À surveiller

Le héros est à 0 : aucun nouveau tour ne doit démarrer.

3. Sortir volontairement avec break

break quitte immédiatement la boucle en cours.

On peut s'en servir pour la fuite.

Une variable booléenne peut mémoriser cette décision pour choisir le message final.

not inverse un booléen.

```
fuite = False

while not fuite:

    fuite = True

    break

print("Sortie")
```

Résultat attendu

Sortie

À surveiller

break quitte uniquement la boucle qui le contient, pas toutes les boucles imbriquées.

4. Limiter les points de vie

Après un soin, il faut vérifier que les PV ne dépassent pas 100.

min renvoie la plus petite des valeurs données ; max la plus grande.

```
pv = 95

pv = min(100, pv + 10)

print(pv)

print(max(0, -5))
```

Résultat attendu

100
0

À surveiller

Plafonne le soin avant l'attaque du monstre. Un héros peut ensuite redescendre à 80 après la riposte.

5. Respecter l'ordre d'un tour

Le héros agit d'abord.

Ensuite, vérifie si le monstre vit encore avant de le faire riposter.

Il faut distinguer une action valide d'une saisie inconnue : cette dernière ne déclenche pas de tour.

```
pv_monstre = 20

pv_monstre = pv_monstre - 20

if pv_monstre > 0:

    print("Riposte")

else:

    print("Monstre vaincu")
```

Résultat attendu

Monstre vaincu

À surveiller

Ne place pas l'attaque du monstre avant le contrôle de ses PV : un ennemi vaincu ne riposte pas.

Exercices intermédiaires

1. Échauffement A · compte à rebours

Avec while, affiche 3, 2, 1, puis « Départ ».

```
compte = 3

# TODO : boucle qui diminue compte

print("Départ")
```

Objectif de vérification

3
2
1
Départ

Indice : Affiche compte puis retire 1 dans la boucle.

2. Échauffement B · soin sans débordement

Un héros possède 97 PV. Ajoute 10 PV et limite le résultat à 100. Fais aussi le test avec 50 PV.

```
pv = 97

# TODO : soigner et plafonner

print(pv)
```

Objectif de vérification

97 → 100 ; 50 → 60

Indice : Utilise un if après l'addition, ou `min(100, pv + 10)`.

Mission à réaliser

Ce que tu dois obtenir

Deux blocs successifs : le mini-jeu d'échauffement puis un combat indépendant. Dans les saisies du navigateur, prévoir aussi les réponses de l'échauffement, par exemple fuir pour le quitter.

1. Échauffement dans Mon travail : crée un mini-jeu indépendant avec vie = 100. attaquer enlève 20 PV au héros, défendre rend 10 (plafond 100), rien enlève 5, fuir sort. Affiche la vie après chaque action et termine si vie ≤ 0.
2. Après l'échauffement, initialise un nouveau combat : pv_hero = 100 et pv_monstre = 100. Ces valeurs ne dépendent pas de l'échauffement.
3. Répète tant que les deux PV sont > 0. Demande exactement attaquer, défendre, rien ou fuir. Ici, attaquer ne retire pas directement de PV au héros.
4. Action du héros : attaquer lance un dé de 1 à 20, et enlève 20 PV au monstre si le dé > 10 ; défendre soigne de 10, plafond 100 ; rien ne fait rien ; fuir termine le combat.
5. Après attaquer, défendre ou rien, si le monstre est encore vivant, lance son dé. Il retire 20 PV au héros si le résultat > 10. Pour une action inconnue : affiche un message et redemande sans riposte.
6. Affiche les deux PV après chaque tour. À la fin, affiche une seule issue : Victoire, Défaite ou Fuite. Teste les cas proposés puis exporte.

Tests à effectuer

- Dé 10 → échec ; dé 11 → 20 dégâts ; héros 95 + soin → 100 avant riposte.
- Monstre à 20, attaque du héros réussie → victoire et aucune riposte.
- Héros à 20, attaque du monstre réussie → défaite ; fuir → fin immédiate.
- Action xyz → message puis nouvelle saisie, PV inchangés.

Gérer les monstres avec une liste

Ajouter, retirer et choisir des éléments sans perdre l'état du jeu.

1. Créer une liste et lire un élément

Une liste conserve plusieurs valeurs dans un ordre.

Les éléments sont séparés par des virgules, entre crochets.

Le premier élément a l'indice 0 ; len donne le nombre d'éléments.

```
sac = ["potion", "cle", "carte"]  
  
print(sac[0])  
  
print(len(sac))
```

Résultat attendu

```
potion  
3
```

À surveiller

sac[3] serait hors de la liste : les indices valides sont ici 0, 1 et 2.

2. Ajouter et retirer

append ajoute à la fin.

remove retire la première occurrence de la valeur demandée.

Vérifie avec in si cette valeur est présente avant de la retirer.

```
sac = ["cle"]  
  
sac.append("potion")  
  
if "cle" in sac:  
    sac.remove("cle")  
  
print(sac)
```

Résultat attendu

```
['potion']
```

À surveiller

remove sur un élément absent provoque une erreur. On retire une valeur, pas un indice.

3. Choisir sans tirer dans une liste vide

random.choice choisit un élément parmi les positions de la liste.

Une liste vide vaut faux dans un test ; if monstres protège donc l'appel à choice.

```
import random

monstres = []

if monstres:

    print(random.choice(monstres))

else:

    print("Forêt libérée !")
```

Résultat attendu

Forêt libérée !

À surveiller

Après la défaite du dernier monstre, il faut tester la liste avant de choisir un nouvel adversaire.

4. Nouvelle règle de combat

À partir de cette partie, on remplace le test > 10 .

L'attaque touche si son dé est $\leq$ attaque.

Le défenseur bloque si son dé est $\leq$ défense.

Les dégâts arrivent seulement si l'attaque touche ET si la défense ne bloque pas.

```
attaque = 12

defense = 7

de_attaque = 12

de_defense = 8

print(de_attaque <= attaque and de_defense > defense)
```

Résultat attendu

True : cette attaque inflige 20 dégâts.

À surveiller

Avec un dé de défense égal à 7, il n'y a pas de dégâts. La valeur limite compte comme une défense réussie.

5. Garder l'état entre les combats

Initialise les PV du héros avant la boucle des monstres.

Initialise les PV du nouveau monstre à l'intérieur.

Le héros conserve ainsi sa fatigue, tandis que chaque nouveau monstre arrive avec tous ses PV.

```
pv_hero = 80

monstres = ["gobelin", "troll"]

monstres.remove("gobelin")

print(pv_hero)

print(monstres)
```

Résultat attendu

```
80
['troll']
```

À surveiller

Ne recrée pas la liste complète au début de chaque tour : les ennemis vaincus reviendraient.

Exercices intermédiaires

1. Échauffement A · sac à dos

Crée une liste contenant cle, ajoute potion puis affiche la liste. Retire ensuite cle sans erreur.

```
sac = ["cle"]

# TODO : ajouter potion

# TODO : retirer cle si présente

print(sac)
```

Objectif de vérification

```
['potion']
```

Indice : Utilise append, puis if ... in ... et remove.

2. Échauffement B · dernier ennemi

Retire troll de la liste puis affiche « Victoire » si elle est vide. Ne fais aucun tirage.

```
monstres = ["troll"]

# TODO : retirer le troll

# TODO : tester la liste vide
```

Objectif de vérification

Victoire

Indice : not monstres est vrai si la liste est vide ; len(monstres) == 0 fonctionne aussi.

Mission à réaliser

Ce que tu dois obtenir

Le menu de courses, puis une exploration de trois combats. Quitte les courses avec q avant de préparer les actions de combat dans les saisies.

1. Échauffement : crée une liste de courses vide et un menu a / r / l / q. a demande un article et l'ajoute ; r demande un article et le retire si présent ; l affiche ; q quitte cet échauffement.
2. Dans la partie 5, recopie le combat de la partie 4 sans effacer la partie 4. Crée monstres = ["gobelin", "araignee", "troll"]. Initialise le héros une seule fois à 100 PV.
3. Tant qu'il reste des monstres et que le héros vit, choisis un ennemi avec random.choice et donne-lui 100 PV. Le héros garde les PV du combat précédent.
4. Remplace la règle du dé > 10 par attaque/défense : héros (12,10), monstre (9,7). Une attaque qui touche et n'est pas bloquée retire 20 PV. Applique cette règle aux deux côtés.
5. Remplace defendre par potion : gain aléatoire de 10 à 20, plafond 100, puis riposte du monstre vivant. Les potions sont illimitées ici. Garde attaquer, rien et fuir ; une saisie inconnue ne consomme pas de tour.
6. Retire le monstre uniquement après sa défaite. Choisis ensuite un autre ennemi seulement s'il en reste. La liste vide donne « Victoire de la forêt ». Une fuite ou une défaite termine toute l'exploration.
7. Teste la dernière victoire, les seuils de défense et la conservation des PV, puis exporte.

Tests à effectuer

- Retirer un article absent → message, pas d'erreur.
- Attaque 12, dé 12 → touche ; dé 13 → rate. Défense 7, dé 7 → bloque ; dé 8 → dégâts.
- Dernier monstre vaincu → liste vide et victoire, aucun choice([]).
- Potion à 95 PV avec gain 20 → 100 avant la riposte ; héros non réinitialisé entre deux monstres.

Construire le bestiaire avec des dictionnaires

Associer des clés à des valeurs et séparer modèles et personnages.

1. Lire une valeur grâce à sa clé

Un dictionnaire associe des clés à des valeurs.

On écrit des paires clé: valeur entre accolades.

Ici les clés sont des textes ; le nom de la clé indique le sens de la donnée.

```
hero = {"nom": "Lina", "pv": 100}

print(hero["nom"])

print(hero["pv"])
```

Résultat attendu

Lina
100

À surveiller

hero[0] ne désigne pas la première valeur. Un dictionnaire se lit par clé, pas par position.

2. Ajouter, modifier, supprimer

Affecter une valeur à une nouvelle clé ajoute cette entrée.

Affecter une valeur à une clé existante la modifie.

del supprime une entrée.

in teste la présence d'une clé.

```
hero = {"pv": 100}

hero["attaque"] = 12

hero["pv"] = 80

del hero["attaque"]

print(hero)
```

Résultat attendu

{'pv': 80}

À surveiller

Accéder à une clé absente produit une erreur. Vérifie sa présence si elle vient d'une saisie.

3. Imbriquer pour décrire plusieurs monstres

Un dictionnaire peut contenir d'autres dictionnaires.

La première paire de crochets choisit le monstre ; la seconde choisit une de ses caractéristiques.

```
bestiaire = {  
    "gobelin": {"pv": 60, "attaque": 9},  
    "troll": {"pv": 100, "attaque": 8}  
}  
  
print(bestiaire["troll"]["attaque"])
```

Résultat attendu

8

À surveiller

Les nombres sont des valeurs, pas des textes : écris 100 sans guillemets si tu veux faire des calculs.

4. Séparer le modèle et le combattant

Avec monstre = modele, les deux noms désignent le même dictionnaire.

copy() crée un nouveau dictionnaire pour le combat.

La copie est superficielle : elle suffit ici car les valeurs du modèle sont des nombres ou des chaînes.

```
modele = {"pv": 60}  
  
monstre = modele.copy()  
  
monstre["pv"] = 40  
  
print(monstre["pv"])  
  
print(modele["pv"])
```

Résultat attendu

40

60

À surveiller

Sans copy(), blesser le combattant modifierait aussi ses PV de départ dans le bestiaire.

5. Choisir parmi des clés

`list(dictionnaire)` construit la liste de ses clés.

Pour valider un choix, teste si le texte saisi est une clé du dictionnaire.

Redemande tant que ce n'est pas le cas.

```
heros = {"mage": 14, "guerrier": 12}

print(list(heros))

print("dragon" in heros)
```

Résultat attendu

```
['mage', 'guerrier']
False
```

À surveiller

Dans le jeu final, copie le dictionnaire du héros choisi avant de modifier ses PV.

Exercices intermédiaires

1. Échauffement A · panneau du bestiaire

Crée le dictionnaire indiqué, puis ajoute la clé `defense` avec la valeur 7. Affiche seulement cette défense.

```
gobelin = {"pv": 60, "attaque": 9}

# TODO : ajouter puis afficher defense
```

Objectif de vérification

```
7
```

Indice : `gobelin["defense"] = ...` ajoute la clé.

2. Échauffement B · garde le modèle neuf

Complète pour que le monstre perde 20 PV sans changer le modèle.

```
modele = {"pv": 60}

# TODO : copier dans monstre

# TODO : retirer 20 PV au monstre

print(modele["pv"])
```

Objectif de vérification

Le modèle doit encore afficher 60.

Indice : Utilise `.copy()` avant de modifier `monstre["pv"]`.

Mission à réaliser

Ce que tu dois obtenir

Le bestiaire de descriptions, puis le jeu complet utilisant les dictionnaires. Le héros sélectionné peut maintenant changer les caractéristiques du combat.

1. Échauffement : crée trois descriptions dans un dictionnaire. Affiche l'une d'elles. Ajoute dragon, modifie sa description puis supprime cette entrée avec `del`.
2. Construis bestiaire avec les champs pv, attaque, defense, description. Valeurs (PV, attaque, défense) : goblin (60,9,7), araignee (40,11,5), troll (100,8,9).
3. Construis heros avec pv, attaque, defense, description et image. Valeurs : mage (100,14,6), guerrier (100,12,10), archer (100,13,8). image contient seulement un nom de fichier, par exemple mage.png ; aucune image n'est chargée par le code console.
4. Demande mage, guerrier ou archer. Tant que le choix n'est pas une clé de heros, redemande. Copie ensuite le héros sélectionné avec `.copy()`.
5. Recopie le combat de la partie 5 dans cette partie. Remplace les caractéristiques fixes par les valeurs des dictionnaires. Initialise les ennemis restants avec `list(bestiaire)`. À chaque rencontre, copie le modèle du monstre choisi.
6. Conserve les règles de la partie 5 : `test <= attaque`, `blocage <= defense`, dégâts 20, potion 10 à 20 plafonnée à 100, pas de riposte après une victoire, fuite possible et PV du héros conservés.
7. Teste un choix inconnu, les trois monstres et l'absence de modification des modèles. Exporte ton fichier final en conservant les six parties.

Tests à effectuer

- Héros dragon → nouvelle demande ; mage → attaque 14, défense 6.
- Goblin 60, dégâts 20 → monstre courant 40, modèle du bestiaire toujours 60.
- Ajout puis suppression de dragon → les trois descriptions d'origine restent.
- Vaincre les trois monstres → victoire ; les PV du héros persistent, les caractéristiques changent selon le monstre.